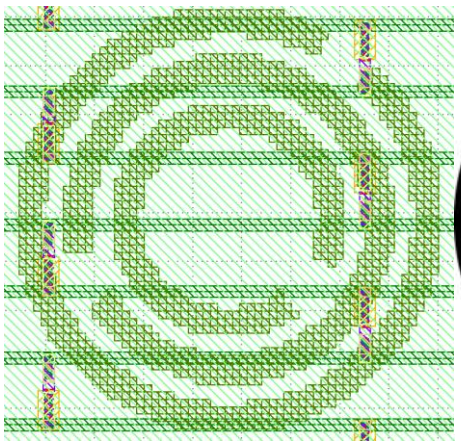
Reverse Engineering a Jane Street ASIC from GDS

Alexander Smallwood

1. First Inspection in KLayout

Jane Street's "Can You Reverse Engineer an ASIC?" provided a GDS layout of a small ASIC. The challenge asked partcipants to recover its netlist, work out the circuit's purpose, find an input that drives "success" high and recover the final string.

To do this, I had to work backwards from the given physical layout. As a complete beginner to ASICs, I downloaded KLayout ana short tutorial to become familiar with the software. When I first opened puzzle.gds, I noticed a particularly dense green section. Zooming in revealed what appeared to be the Jane Street Logo. Easter egg number 1?

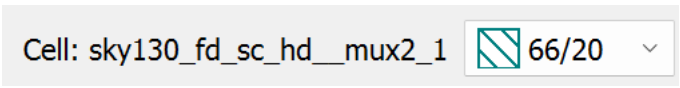


KLayout

Jane Street logo

2. Identifying SKY130 standard cells

After discovering the easter egg I moved onto the main objective. By clicking through instances in puzzle.gds, I found names such as sky130_fd_sc_hd_and2_2 and sky130_fd_sc_hd_dfrtp. At first these meant absolutely nothing to me



Researching the names led me to the sky130_fd_sc_hd library, which the SKY130 documentation identifies as the High Density Standard Cell Library provided for SKY130 designs (SkyWater PDK Authors, n.d-a).

Translating the cell names

I used the official cell definitions from the cell hierarchy in KLayout to translate the unfamiliar names into logic that I recognised form brief fpga research. (SkyWater PDK Authors, n.d.-b)

Cell name	Function
sky130_fd_sc_hd__and2_2	2 input AND gate
sky130_fd_sc_hd__and2b_2	2 input AND, one input inverted
sky130_fd_sc_hd__nand2_2	2 input NAND gate
sky130_fd_sc_hd__xor2_2	2 input XOR gate
sky130_fd_sc_hd__or2_2	2 input OR gate
sky130_fd_sc_hd__inv_2	Inverter
sky130_fd_sc_hd__mux2_1	2:1 multiplexer

This list essentially tells us which cell types exist but with no order, like a lego but without the instructions.

I then looked up the specific dfrtp, dfctp and dfstp cells. Their invidiual functional models identifiy them as flip-flops, meaning the ASIC contained stored state and was not purely combinational (SkyWater PDK Authors, n.d.-b).

3. Automating the Cell Inventory

Instead of ensuing the laborious task of clicking around the chip we can use a KLayout macro in Python to ask what and where the cells are in the design. To do this I prompted ChatGPT to create a macro to get the top level cell, and then print what type of cell it is and where that cell is.

Macro: "firstcellfinder"

This output contained: 9878 lines, of those lines 1618 Sky130 standard cell instances, majority of those are non logic. Of course, if only a minority of what printed is actually logic I prompted ChatGPT to refine the script to only print logic gates and registers.

Macro: refinedcellfinder

Observed Group	Count	Interpretation
Dfrtp, dfxtp, dfstp	92	Flip flops: 84 + 4 + 4
Clkbuf variants	32	Possible clock distribution sturcutre
Mux 2 variants	21	Selection logic

The puzzle hints that the physical arrangement of the puzzle can give clues. So the next step is to identify the physical arrangement of all of the aforementioned cells.

Macro: cellarrangementfinder

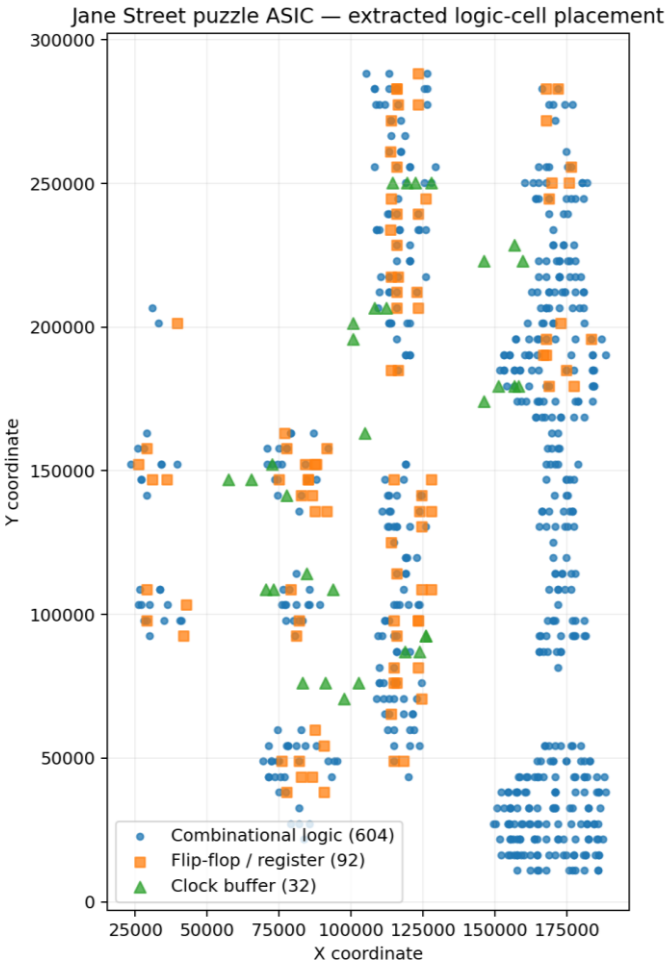
This gives us 728 logic instances with coordinates, 92 being flip flops.

From the output I asked GPT to generate a map of the circuit.

This map gives us notable observations.

1. The far right of the map shows a huge logic heavy vertical region
2. Middle shows a nother vertical structure with lots of flip flops
3. Left shows smaller "islands" of flip flops

However we must now recover the wires between the gates.



4. Locating Pins and Routing Geometry

To follow a wire, I first needed a precise starting point. I opened the sky130_fd_sc_hd__dfrtp_2 definition and printed its text shapes across the layout layers. The official model identifies this cell as a positive-edge-triggered D flip-flop with an active-low asynchronous reset, while the library documentation explains the high-density cell dimensions (SkyWater PDK Authors, n.d.-a; n.d.-b).

```
Cell: sky130_fd_sc_hd__dfrtp_2
TEXT 67 5 Q r0 8965,2210
TEXT 67 5 Q r0 8965,1870
TEXT 67 5 Q r0 8965,1530
TEXT 67 5 Q r0 8965,510
TEXT 67 5 RESET_B r0 7490,850
TEXT 67 5 D r0 1610,1530
TEXT 67 5 CLK r0 235,1530
TEXT 67 5 CLK r0 235,1190
TEXT 67 5 RESET_B r0 7490,1190
TEXT 68 5 VGND r0 230,0
TEXT 68 5 VPWR r0 230,2720
TEXT 64 5 VPB r0 230,2720
TEXT 64 59 VNB r0 230,0
TEXT 83 44 dfrtp_2 r90 0,0
```

Macro: pindeftest

This revealed repeated labels for D, Q, CLK and RESET_B at coordinates local to the cell definition. Those local coordinates could not be compared directly with top-level routing because placed cells may be translated, rotated or mirrored.

Macro: transformationrotation

For the placed flip-flop at (167900, 282880), the macro applied an instance transformation to every local pin coordinate. KLayout's Trans object represents the cell's rotation or mirror operation and displacement, and its transformation operation maps a point into the parent coordinate system

```
Q local = 8965,2210 global = 176865,285090 layer = 67/5
Q local = 8965,1870 global = 176865,284750 layer = 67/5
Q local = 8965,1530 global = 176865,284410 layer = 67/5
Q local = 8965,510 global = 176865,283390 layer = 67/5
RESET_B local = 7490,850 global = 175390,283730 layer = 67/5
D local = 1610,1530 global = 169510,284410 layer = 67/5
CLK local = 235,1530 global = 168135,284410 layer = 67/5
CLK local = 235,1190 global = 168135,284070 layer = 67/5
RESET_B local = 7490,1190 global = 175390,284070 layer = 67/5
```

This we now have a way to find the cell type, placement, local pins, chip coordinates.

Now to do this for every gate.

Macro: layeroverlap

It should also be mentioned that layer 67/5 is a label layer NOT A METAL LAYER. The actual geometry we need is most likely on the corresponding local interconnect drawing layer, then through contacts into Metal 1 and so forth. However I relaised that this could also yield noise so I also ensured to only include layers relevant to routing, being the metal, vias and label 1.

The result is a relatively large output that shows the chosen pin lying on conductive shapes. Now that we know we can identify routing geometry specific to a pin we can trace routing electrically connected to that pin.

5. Recovering the first complete net

Macro:autowirefollower

I then built merged regions for li1 and metal layers 1–5, plus the contacts and vias between adjacent layers. Beginning at the D-pin coordinate, the macro expands through any conductive shape joined by a via. KLayout's Region API supplied the merged and interacting geometry operations used for this trace

```
=== CONNECTED ROUTING ===
li1 area = 1620025 bbox = (167960,280415;169775,285330)
met1 area = 319200 bbox = (168445,281900;169670,283860)
met2 area = 359600 bbox = (169380,281870;169640,283890)

=== CONNECTED PINS ===
D | sky130_fd_sc_hd__dfrtp_2 | instance r0 167900,282880 | pin position 169510,284410
X | sky130_fd_sc_hd__or2_2 | instance m0 166520,282880 | pin position 168595,281010

Total unique pins found: 2
```

The result is a fully recovered net! Essentially, sky130_fd_sc_hd__or2_2 instance m0 166520,282880 pin X connects to sky130_fd_sc_hd__dfrtp_2 instance r0 167900,282880 pin D. Super cool!

Now to automate every signal pin in every date! GPT also suggested we deduplicate the nets.

Macro: autowirefollowereverywhere

The result is a .txt file noted by "recovered_jane_street_netlist.txt" which contains 728 instances, 2767 logical pins and 739 recovered nets. The clock structure also matches the physical clock structure identified on the map. NET_74 shows the global reset distribution, it connects the active low reset of the dfrtp registers.

However multiple nets have no obvious output driver. This is not necessarily wrong though, they can be potentially described as top level puzzle inputs that enter the design via external routing rather than a standard cell output.

6. Classifying the Recovered Netlist

Macro: netlistanalyser

The result is a file noted by "jane_street_interface_report.txt, which found 739 nets, 5 no driver candidates, 29 output only nets, 33 clock related nets, 1 reset/set net and zero multiple driver nets. This confirms the consistency of my connection extractions.

Classification	Count	What it suggested
No internal driver	5	Possible external inputs
One Driver, no internal loads	29	Possible top level outputs
Clock related	33	Clock distribution and source
Reset/set	1	Shared control net NET_74
Multiple internal drivers	0	No obvious extraction conflict

Five No Driver Candidates

Net	Observed loads	Working interpretation
NET_74	Reset/set pins	External reset or set control
NET_274	U194.A on clkbuf_116	Possibly master clock input
NET_15	Large fan out	External control or data input
NET_395	And2b input	External input candidate
NET_590	Two compound gate inputs	External input candidate

These interpretations were only hypotheses at this stage based on the fan out and destination pins, however they became stronger when I proved how those nets were used.

7. Identifying a Candidate Eight Bit output

Furthermore, of the 29 output only nets, most do not feed another standard cell pin.

Candidate net	Driver	Candidate net	Driver
NET_66	U24.X	NET_68	U25.X
NET_81	U30.X	NET_83	U31.X
NET_113	U46.X	NET_114	U48.X
NET_124	U61.X	NET_158	U89.X

These are all AND3 outputs which of course is an 8-bit output bus, exactly the sort of thing used to emit charcaters/bytes!

Thus I can hypothesize: The 8 AND3 outputs may be the challenges output byte! This is further solidified by the fact that we are told that that the puzzle produces an output string so the an 8 bit physically grouped output fits very nicely.

It makes sense that the next thing to do is figure out what drives the inputs of the AND3 outputs. This can help solidify the 8 bit output bus thesis. So I prompted ChatGPT to make a script that traces backwards from the 8 AND3 outputs.
Macro: and3tracer

The output is a file named “output_bus_trace.txt”. It tells us that all eight AND3 outputs have the exact same structure, being: OUT[i] = NET_45 AND DATA[i] AND NET_47. Where:

Output	Translation
NET_45 = U27.Q	flip-flop state
NET_47 = U49.Y	shared combinational enable

And all DATA 0 to 7 = U32.X, U44.X, U94.X, U33.X, U82.X, U23.X, U93.X, U63.X

Thus every output AND gate shares NET_45 and NET_47 while its B input is unique, this is repeated across all eight outputs. This infers that NET_45 and NET_47 are likely some form of output_valid, output_enable, state_is_outputting, signals.

Therefore, the eight 031A gates generate an 8-bit data value, and the eight AND3 gates output when the circuit wishes to emit it.

8. Tracing the Byte-Generation Logic

Macro: 031Atracer.lym

The eight data sources were all o31a_2 cells. The official functional model defines o31a as a three-input OR feeding a two-input AND: X = (A1 OR A2 OR A3) AND B1 (SkyWater PDK Authors, n.d.-d).

All eight gates shared NET_61 as one input. The remaining inputs differed by bit, so I needed to walk backwards through the surrounding compound logic rather than analyse each gate in isolation. The later appearance of a22o gates was interpreted using the official equation X = (A1 AND A2) OR (B1 AND B2) (SkyWater PDK Authors, n.d.-e).

MACRO: eightbytewalkback.lym

The recursive walk stopped whenever it reached a flip-flop, a constant cell or a no-driver net. This reduced a large combinational network to a smaller set of state sources that could be investigated directly.

Source Set	Registers	Observed role

Shared Pair	U28,U29	Reached from multiple byte bits
Four state group	U35 through to U38	Four shared state signals
Additional shared group	U439, U441, U443, U445, U446, U451, U452, U453	Reached though deeper logic
Bit-specific group	U245 through to U248, U256 through to U259	One register mapped to each byte bit

*register sources found by byte walk back

Bit	Register	Bit	Register
0	U258	4	U248
1	U256	5	U246
2	U259	6	U247
3	U257	7	U245

*One-to-one mapping between candidate byte bits and bit-specific registers.

Four shared state bits could represent up to 16 states, but at this point it was too early to call them an address or a complete state machine. I therefore inspected their next-state inputs and reset behaviour before assigning a function. The external candidate NET_590 appeared only in the logic for bits 4 and 7, another clue to preserve for later testing.

9. Investigating the State Registers

Macro: interestingregisters.lym

This macro inspected U35–U38 and the eight bit-specific registers, then printed the drivers of each D, CLK, RESET_B and SET_B pin. It saved the result as interesting_register_inputs.txt.

The D inputs of U35–U38 were driven by combinational logic, so they could not yet be described as an ordinary binary counter. The bit-specific registers all used NET_74 for reset or set control, but the polarity differed across the byte.

Candidate Bits	Registers	Reset Action	Q after reset
0-3	U258, U256, U259, U257	SET	1
4-7	U248, U246, U247, U245	RESET	0

Reading bits 7 down 0 gave 00001111 = 0x0F, I thought this was an ASCII character, but it is not meaning these eight registers are probably not the output byte register, rather they may be eight state/history bits that individually participate in producing the output byte.

Macro: fourstateequationsexposed.lym

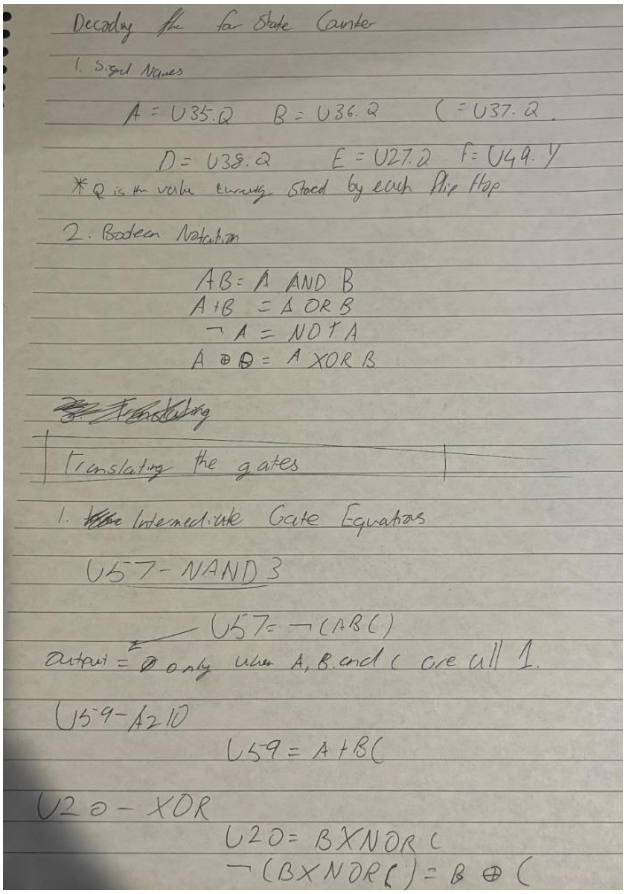
The macro identified the gates driving the four D inputs and saved their immediate input nets and drivers as state4_next_logic.txt. The paths through U18, U47 and U22 depended on NET_45 from U27.Q; U18 and U47 also shared NET_47 from U49.Y. U35 followed a different path involving U38.Q, U57, U59 and U27.Q. This showed coupled next-state logic, but not yet a complete or proven state-machine interpretation.

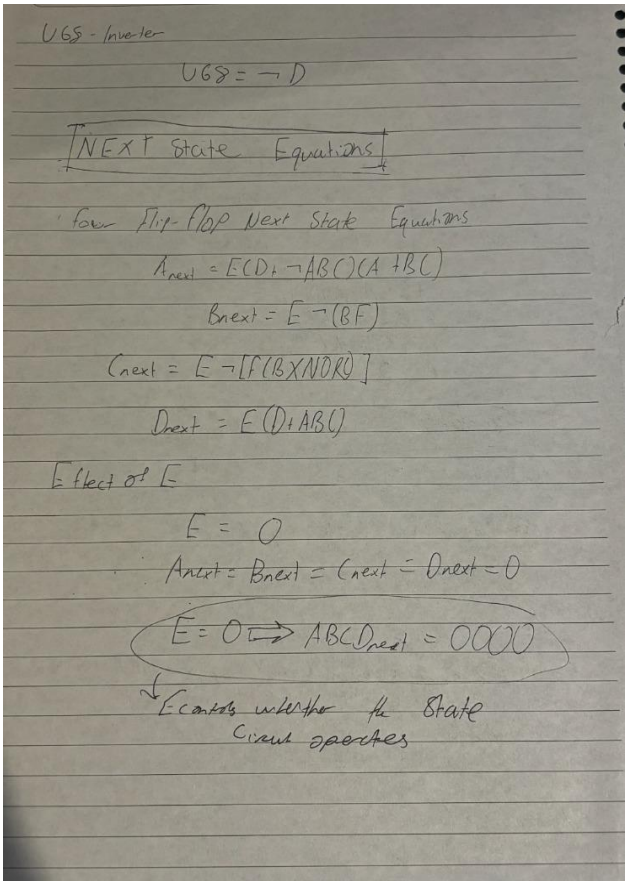
Macro: 4statelogic.lym

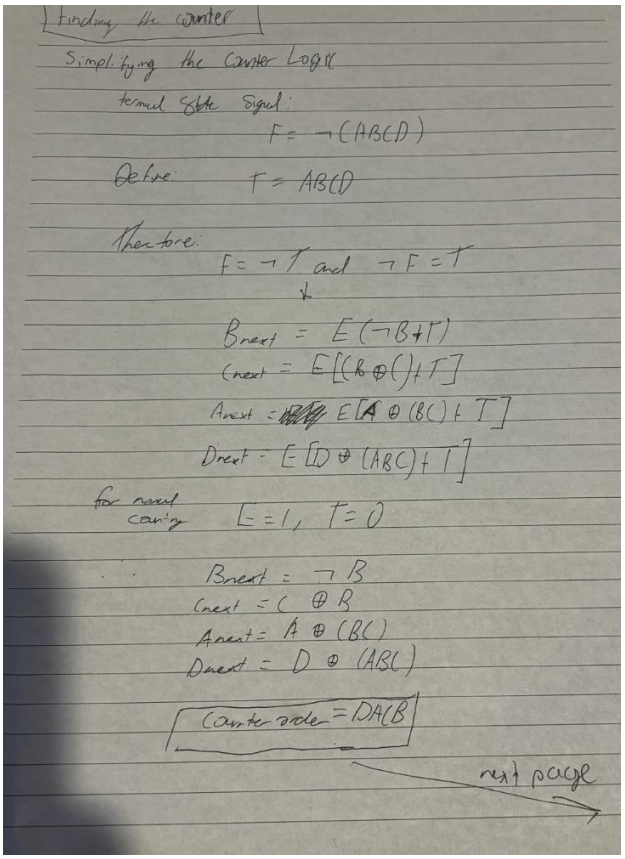
The next macro expanded U57, U59, U20 and U68, reporting each output net together with every input net and driver. It saved this intermediate layer as state4_intermediate_logic.txt. Thus the next task was to convert these traced connections into explicit Boolean equations.

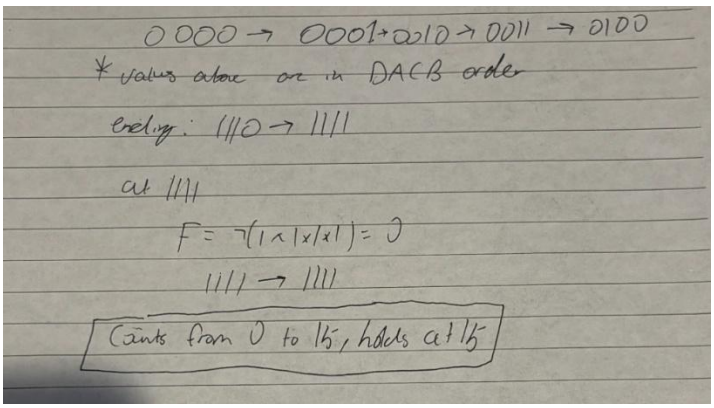
10.Boolean Analysis of the Four-Bit State Counter

The Boolean equations revealed that the four flip-flops formed a saturating four-bit counter in the reordered bit sequence DACB, allowing 15 bytes to be output before the counter reached and held its terminal state.









11. Separating the Output Sequencer from the Checker

Macros: u343tracer.lym; u350u473u352expansion.lym; whatcalcseightcheckerbits.lym; upperlowechecker.lym; lastlittleexpansion.lym

I thought U343 was the success latch at first since it rose when the input phase finished and stayed high. But it turns out it's just a completion trigger. Once it goes high it flips on U27, which turns on the output-sequencing subsystem. Expanding U350, U473 and U352 gave me two groups of four flip-flops. The lower group (U347, U345, U355, U348) counts 0 to 10 and wraps. The upper group (U459, U460, U467, U463) advances each time the lower one wraps, so together that's 11x11, 121 output positions. So the checker finishes first and raises U343, and only then does U27 latch high and let the counters step through output. These counters just generate output, so they're not part of the secret-input test, which matches the earlier hint about one block being pure output with no effect on success.

Counter Role	Registers	Observed Behaviour
Lower Digit	U347, U345, U355, U348	0 to 1 through to 10 to 0
Upper Digit	U459, U460, U467, U463	Advances when lower digit wraps

12.Recovering the Top Level Interface

Macros: shortestpath.lym, toplevellabelextract.lym, measured.lym

Before I knew the external pins, I ran a few forward-trace experiments to try to guess them. NET_15 reached 78 registers and all eight output bits, so it clearly had broad influence over serial data, but it wasn't on my original list of candidates for the success register. NET_590 reached zero registers and only two output bits, which suggested it was more about output generation than the checker. NET_395 reached all 92 flip-flops in the sequential network, so it looked like a likely enable or advance signal, though reachability alone didn't confirm what it actually did. The real answer came from inspecting the text labels still sitting in the top-level GDS cell, which exposed the external interface directly: I, enable, clk, rst_n, success and O[7:0]. Measuring the routing showed I tied to NET_15, enable to NET_395, clk to NET_272 and rst_n to NET_74. Most importantly, success turned out to be NET_78, driven by U29.Q. So that confirmed U343 was never the real success signal.

Port	Recovered net / Driver
I	NET_15
Enable	NET_395
Clk	NET_272
Rst_n	NET_74
Success	NET_78 = U29.Q
O[0]	NET_68 = U24.X
O[1]	NET_158 = U89.X
O[2]	NET_68 = U25.X
O[3]	NET_114 = U48.X
O[4]	NET_113 = U46.X
O[5]	NET_83 = U31.X
O[6]	NET_124 = U61.X
O[7]	NET_81 = U30.X

13. Tracing the Real Success Condition

Evidence files: successequation.lym, finalsuccesschecks.lym, finalsuccessexpanded.lym, comparatorselectors.lym, smallercrack.lym, next4reg.lym, missedcomparator.lym

Tracing backward from U29 showed that U34 drives U29.D, and U29.Q also feeds back through U34, which creates a holding path so success stays high once it's asserted. The new-success branch only fires when U343 = 1 and U27 = 0, and since U343 causes U27 to rise on the next clock, there's only a one-clock window between the input phase finishing and the output sequencer kicking in. Success also needs the sticky registers U382 and U411 to stay at zero. I first read U382 as a generic bad-1 flag, but targeted tests showed it specifically catches adjacent-1 violations during the input phase. U411 is sticky too, and I couldn't pin down its full role from a single local trace, but simulation later

confirmed it enforces the row and between-row conditions I'd found experimentally. U419 turned out to be a multiplexer choosing between U412 and the real serial input I, which showed both sticky failure paths trace back to the input stream. I also mapped out enable-history chains like U386 -> U387 -> U383 and finished off the rest of the comparator cone before switching from manual expansion to simulation.

14. Building and Validating the Gate Level Simulator

Evidence files: simulator1.lym, sim2.lym

Manual expansion had hit diminishing returns, so the next step was building a real gate-level simulator. simulator1.lym inventoried what I needed to support: 66 unique cell types across 728 relevant instances, including 92 flip-flops. sim2.lym then parsed the recovered netlist, repeatedly evaluated combinational gates until their values settled, and updated every flip-flop together on each rising clock edge, with active-low reset and set behaviour built in. I exposed U29/success and O[7:0] using the physically measured interface mappings. Validation covered checking that every required cell type had an implementation, that no combinational value was left unresolved, that reset produced success = 0, and that the clocked state advanced correctly under a simultaneous-edge model. One assumption mattered a lot here: I set the initial values of the dfxtp cells to zero, since those cells have no asynchronous reset. The later verifier reused this same assumption, so the two programs agreeing doesn't independently prove it.

Model Item	Value Used
dfrtp initial/reset state	0
dfstp initial/set state	1
dfxtp initial state	Assumed 0
I	NET_15
enable	NET_395
rst_n	NET_74
success	NET_78 / U29.Q

15. Discovering the 11 x 11 Input Constraints

Macro: sim2improved.lym, sim2improved+.lym, sim3.lym, sim4.lym, sim5.lym

The first search branched on I = 0 and I = 1, dropped states that set U382 or U411, and merged histories landing on the same 92-flip-flop state. Surviving prefixes went 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, a clean Fibonacci sequence (super cool! Very cool work Anish and Benjamin). It also confirmed a direct search would keep growing exponentially and never scale to 121 bits.

A shorter follow-up explained why the substring "11" is forbidden, as well as why U382 trips on the second 1 of an adjacent pair. There are 233 length-11 strings with no consecutive 1s, and sim3 found 45 of those get accepted, each with exactly two non-adjacent 1s, matching C(11,2) - 10 = 45. sim4 tested all 45 x 45 = 2,025 ordered pairs of legal rows: 420 survived, 1,605 rejected. Turns out a 1 in the next row can't sit in the same or adjacent column to a 1 above it, so combined with the no-consecutive-1 rule per row, no two marked cells can be a king's move apart. sim5 tried treating each legal row as a state and only exploring legal transitions while merging identical ASIC states, but that still blew up, which is what pushed me from enumeration to symbolic solving.

16. Solving the Recovered Circuit with Z3

Macro: z3solver.py

Instead of testing candidate strings one at a time, I turned the 121 unknown serial input bits into Boolean variables and encoded the recovered circuit as logical constraints in Z3. The model covers every supported combinational gate, every flip-flop transition and 124 clock intervals. The first 121 intervals are symbolic, and after that I is held low, enable stays high and rst_n stays deasserted. I also folded in the structure I'd found experimentally, being, exactly two non-adjacent 1s per row, no vertical or diagonal contact between neighbouring rows. Success is allowed to go high at state 121, 122, 123 or 124. NET_590 is fixed to zero and dfxtp's initial state is assumed zero, same as before. Z3

returned one candidate satisfying the gate-level model, the spatial rules and all the stated assumptions, which proves the model is satisfiable but not that the candidate is unique.

Row	11 bit word	Marked Columns
1	00000001010	8, 10
2	10000100000	1, 6
3	00000001010	8, 10
4	10100000000	1, 3
5	00001010000	5, 7
6	00100000100	3, 9
7	00001000001	5, 11
8	01000010000	2, 7
9	00010000001	4, 11
10	00000100100	6, 9
11	01010000000	2, 4

17.Replaying the Candidate and Decoding the Output

Macro: verify_candidate.py

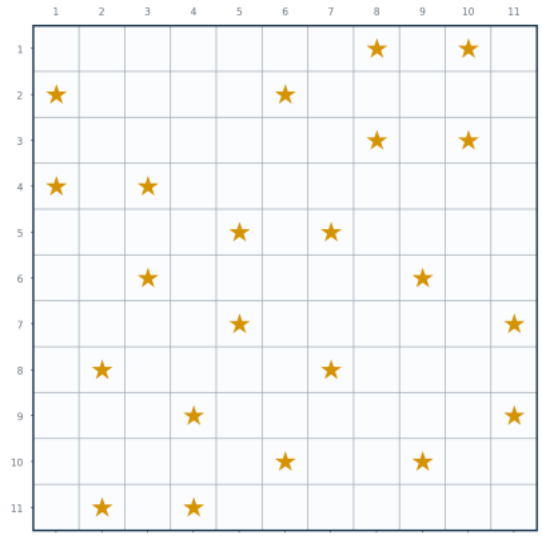
A separate, plain-Python verifier re-parsed the recovered netlist and replayed the fixed Z3 candidate using concrete Boolean values. I applied reset, then fed in the 121 candidate bits, then ran 24 more clocks so the output sequencer could finish. The verifier didn't depend on Z3's symbolic search, but it reused the same netlist, gate interpretations, NET_590 = 0, and the zero-initial-state assumption for dfxtp. So it's a strong consistency check rather than fully independent proof of every modelling choice. During the replay, U382 and U411 stayed zero. U343 rose at state 121. U27 rose at state 122. U29/success rose at state 122 too and held from there. The decoded output bytes gave the intended message. The timing lining up with coherent text is strong evidence the recovered connectivity and state model match the real design, and that the candidate itself is right.

State/register	Observed result
U382	Remained 0
U411	Remained 0
U343	Rose at state 121
U27	Rose at state 122
U29	Rose at state 122 and remained high

Decoded output = (*TWO STARS*)

Woo hoo!

Arranging the 121 serial bits as eleven rows of eleven cells revealed 22 marked cells, every row contains exactly 2 marks. No two marks touch horizontally, vertically or diagonally. This is like a two-star Star Battle style puzzle.



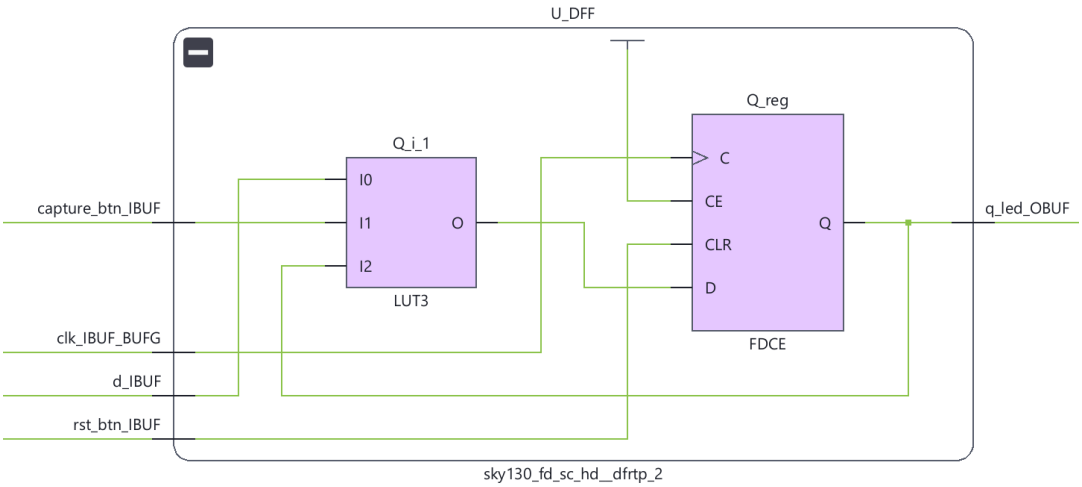
When arranged as a grid

18.Recreating the ASIC on a Basys 3 FPGA

While I was able to find the answer using a software simulation, I wanted an excuse to use my new (and first) FPGA board! Thus, I decided to transplant the recovered gate level logic onto a Basys 3 development board which contains an Artix-7 FPGA. The aim was to preserve the logical cell graph and verify that it behaved the same way when synthesised.

19.Learning the FPGA workflow with small tests

As I had never used an FPGA before I started with a small test to get familiar with the device. I then recreated a SKY130 AND cell and confirmed that vivado mapped it into an FPGA LUT.

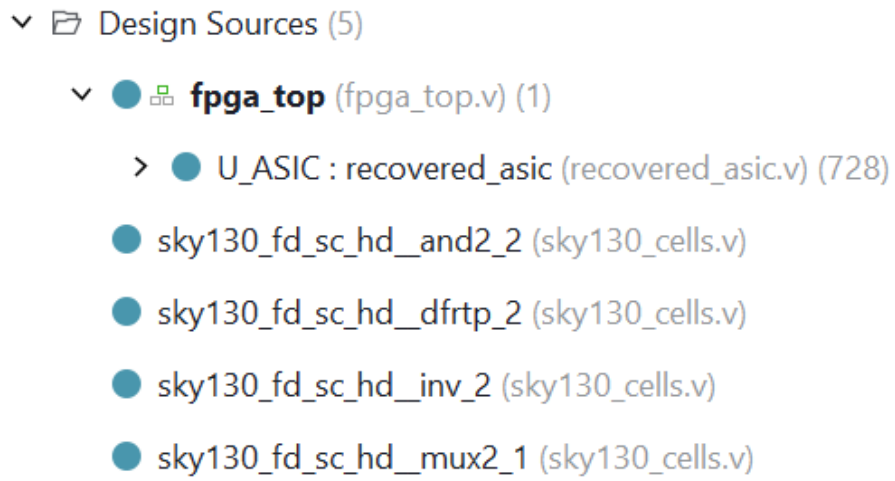


20.Recreating the SKY130 cell library

The recovered design used 66 unique SKY130 cell types. I manually recreated a small set first which included: an AND gate, inverter, 2:1 multiplexer and resettable D flip-flop. I then automated the repetitive part with a Python generator that pulled the official functional definitions and renamed them to the specific names present in the recovered netlist.

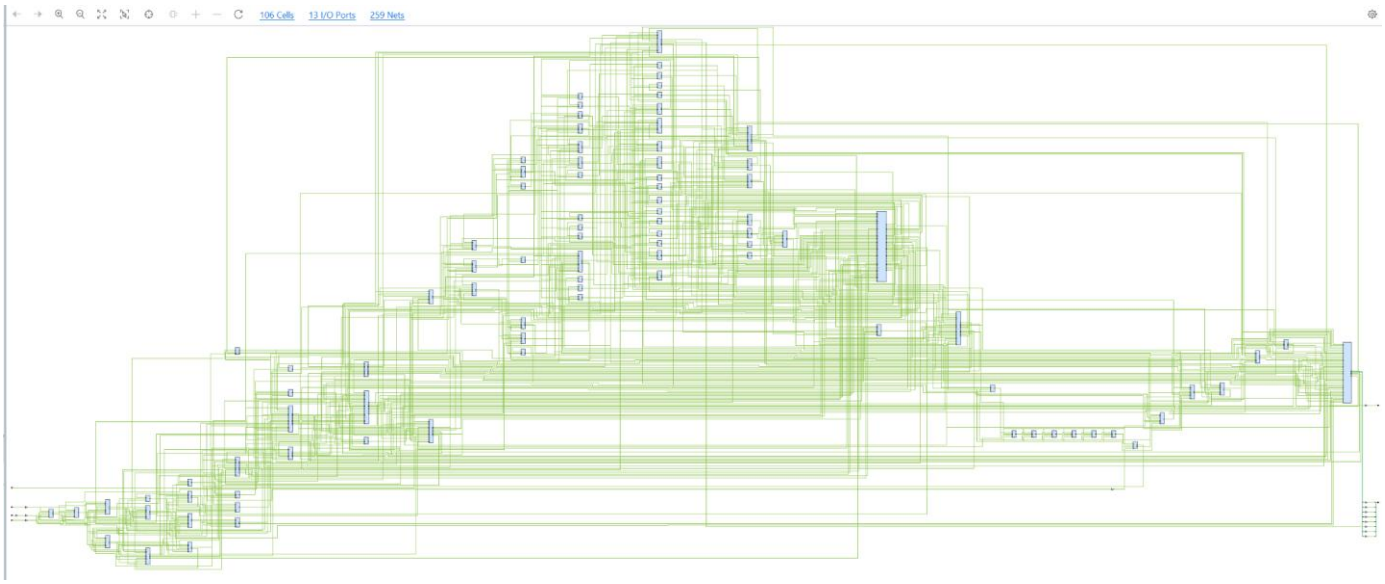
21.Generating the recovered ASIC Verilog

A Python script then automatically recreated the 728 recovered SKY130 cells in Verilog and connected them using the recovered netlist.



22.FPGA synthesis

Vivado successfully synthesized the full recovered design. The original 728 SKY130 instances did not remain as 728 physical FPGA primitives, because synthesis is free to combine Boolean logic into the Artix-7 LUT structure. The synthesized design therefore contained 106 FPGA cells and 259 nets while still implementing the same observable logic.

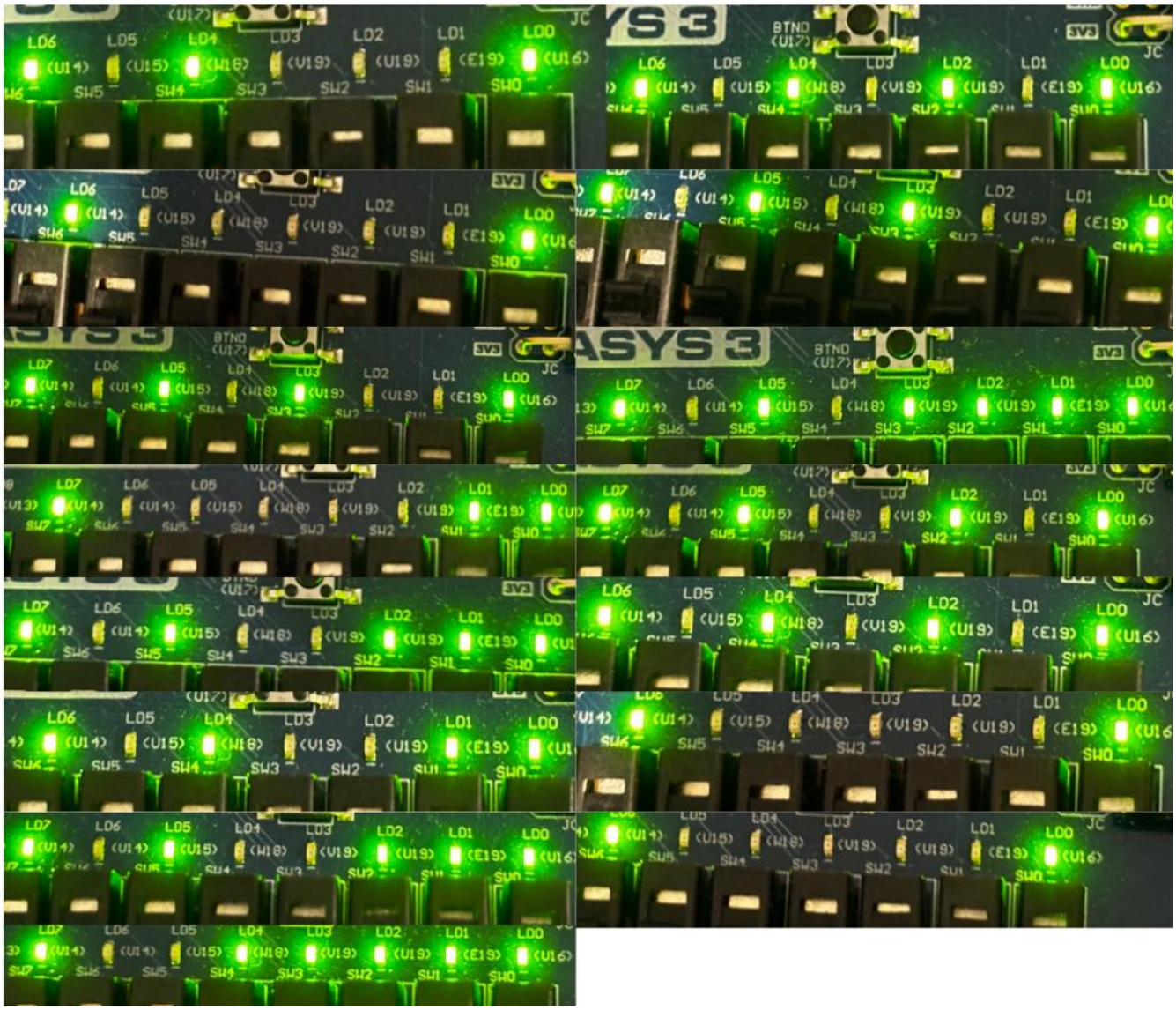


A small wrapper module was used to connect the reconstructed ASIC to the Basys 3 clock, reset, LEDs and USB-UART input without changing the recovered ASIC logic.

The 121-bit solution was then sent from the PC over USB-UART. During testing, the receiver initially counted 124 characters instead of 121, which prevented the checker from succeeding. After fixing the UART receiver, the FPGA correctly received all 121 bits.

23. Final Hardware Verification

After sending the 121-bit solution, the FPGA's success signal activated on the next clock. I then stepped through the ASIC's 8-bit output and converted each LED pattern to ASCII, which reproduced all 15 characters from the software result!



Step	Lit LEDs	Binary O	ASCII
1	LD4, LD6	00101000	(
2	LD2, LD4, LD6	00101010	*
3	LD6	00100000	space
4	LD3, LD5, LD7	01010100	T
5	LD1, LD2, LD3, LD5, LD7	01010111	W
6	LD1, LD2, LD3, LD4, LD7	01001111	O
7	LD6	00100000	space
8	LD1, LD2, LD5, LD7	01010011	S
9	LD3, LD5, LD7	01010100	T
10	LD1, LD7	01000001	A
11	LD2, LD5, LD7	01010010	R
12	LD1, LD2, LD5, LD7	01010011	S
13	LD6	00100000	space
14	LD2, LD4, LD6	00101010	*

15	LD1, LD4, LD6	00101001	)

Thus, it constructs (* TWO STARS *)

References

AMD. (2025, April 1). *7 series FPGAs configurable logic block user guide (UG474)* (Rev. 1.9). https://docs.amd.com/r/en-US/ug474_7Series_CLB

AMD. (2026, July 8). *Vivado Design Suite user guide: Synthesis (UG901)* (Version 2026.1). https://docs.amd.com/r/en-US/ug901-vivado-synthesis

Digilent. (n.d.). *Basys 3 reference manual*. Retrieved September 3, 2026, from https://digilent.com/reference/programmable-logic/basys-3/reference-manual

Microsoft Research. (n.d.). *Introduction*. Online Z3 Guide. Retrieved September 3, 2026, from https://microsoft.github.io/z3guide/docs/logic/intro/

QNFCF. (n.d.). *KLayout tutorial* [Video playlist]. YouTube. Retrieved September 3, 2026, from https://www.youtube.com/playlist?list=PLLncBybQzAvlvDUUwHU9V5FYIEVagN9Wp

Singhani, A., & Devlin, B. (2026, August 5). *Can you reverse engineer an ASIC?* Jane Street Blog. https://blog.janestreet.com/can-you-reverse-engineer-an-asic/

SkyWater PDK Authors. (2020). *SkyWater SKY130 PDK documentation*. https://skywater-pdk.readthedocs.io/en/main/

Thank you for reading (:

I hope you enjoyed as much as I did solving this!